1.

Abstract [3]

The objective of this mini-project is to design and develop a **console-based Flight Reservation System** using **Java** [44], demonstrating the concepts of **Data Structures and Algorithms (DSA)**[555]. This system allows users to manage flight information efficiently with core features such as adding new flights, searching for available flights, sorting them based on price, booking and canceling tickets, and displaying passenger details[6]. The project primarily focuses on applying fundamental **searching and sorting algorithms**—specifically **Linear Search** and **Bubble Sort**—to solve real-world problems effectively, reinforcing the understanding of computational logic and data manipulation[777].

2.

Introduction [8]

In the modern era of automation, efficient management of flight data plays a vital role in travel planning[9]. This project, **Flight Reservation System**, provides a structured and algorithmic way to maintain flight schedules, manage bookings, and access real-time flight details[10]. The implementation demonstrates practical applications of **arrays, classes, searching, and sorting algorithms** [11]—which are essential components of the **Data Structures and Algorithms curriculum** for second-year Computer Engineering students under SPPU (Savitribai Phule Pune University)[12]. The system successfully demonstrates the core principles of DSA by applying these algorithms effectively to real-world data[13].

3.

Objectives [14]

The main objectives of this mini-project are:

- To understand and implement **searching and sorting algorithms**[15].
- To demonstrate the use of **Object-Oriented Programming (OOP)** principles in Java[16].
- To simulate a real-world application using **data structures**[17].
- To enhance problem-solving and **algorithmic thinking skills**[18].
- To analyze the time and space complexity of implemented algorithms[19].

4.

System Requirements [20]

Software Requirements [21]

- **Java, JDK 8+**
- **IDE:** IntelliJ/Eclipse/VS Code
- **Operating System:** Windows/Linux/macOS

Hardware Requirements [22]

- **Processor:** Intel i3+ or equivalent
- **RAM:** 4GB
- **Storage:** 200MB free storage

5.

Algorithms Used [23]

This project utilizes two fundamental algorithms to manage and organize the flight data efficiently:

Linear Search Algorithm (for Flight Search) [24]

This algorithm is used to find flights based on the user's destination. It sequentially checks each element in the flight data structure until a match is found.

- **Steps:**
    - Step 1: Start [25]
    - Step 2: Accept the destination [26]
    - Step 3: Compare sequentially [27]
    - Step 4: Display results [28]
    - Step 5: Stop [29]
- **Time Complexity:** $O(n)$ [30]

Bubble Sort Algorithm (for Sorting Flights by Price) [31]

Bubble Sort is used to arrange the available flights in ascending or descending order of their ticket price, allowing users to quickly find the cheapest or most expensive options.

- **Steps:**
    - Step 1: Start [32]
    - Step 2: Compare adjacent prices [33]
    - Step 3: Swap if needed [34]

- Step 4: Repeat until sorted [35]
- Step 5: Stop [36]

- **Time Complexity:** $O(n^2)$ [37]

6.

Implementation Details and Output Screenshots [38]

The system is modular, implemented in Java, and follows OOP principles. The main features are organized into the following modules[39]:

- **Add Flight:** Allows an administrator to input new flight records (Source, Destination, Price, Capacity).
- **Display Flights:** Shows a list of all current flights.
- **Search Flights:** Uses **Linear Search** to find flights based on the destination input by the user[40].

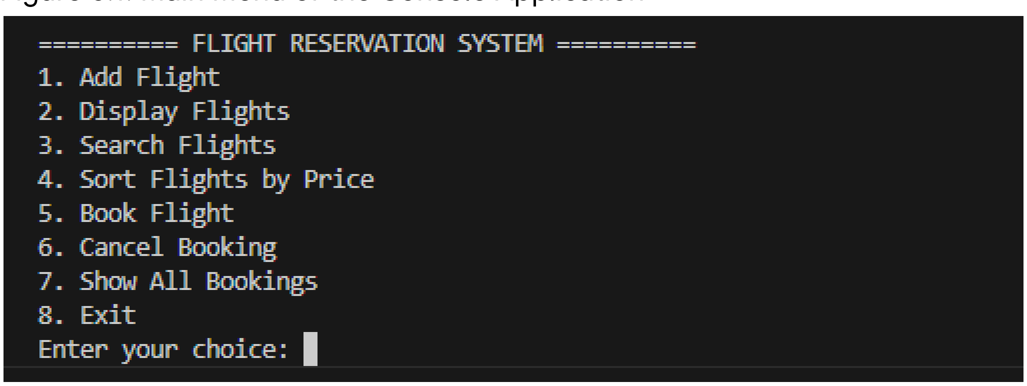
- **Sort Flights:** Uses **Bubble Sort** to arrange flights based on their price[41].

- **Book Flight:** Decrements the available seats for a selected flight and records passenger details.
- **Cancel Booking:** Increments the available seats and removes the passenger record.
- **Show Bookings:** Displays the list of all booked passengers.

Screenshots of Execution and Output

Please paste the screenshots of your project running here. Ensure the screenshots clearly demonstrate the main menu, the search functionality, the sorting functionality, and a successful booking/cancellation.

Figure 6.1: Main Menu of the Console Application



```
========== FLIGHT RESERVATION SYSTEM ==========
1. Add Flight
2. Display Flights
3. Search Flights
4. Sort Flights by Price
5. Book Flight
6. Cancel Booking
7. Show All Bookings
8. Exit
Enter your choice:
```

Figure 6.2: Demonstration of Add Flight and Display Flights Modules

```
========== FLIGHT RESERVATION SYSTEM ==========
1. Add Flight
2. Display Flights
3. Search Flights
4. Sort Flights by Price
5. Book Flight
6. Cancel Booking
7. Show All Bookings
8. Exit
Enter your choice: 1
Enter Flight ID: 106
Enter Source: Pune
Enter Destination: Hyd
Enter Time: 4
Enter Price: 1000
Enter Total Seats: 45
Flight Added Successfully!
```

Figure 6.3: Demonstration of Search Flights (Linear Search) by Destination

```
========== FLIGHT RESERVATION SYSTEM ==========
1. Add Flight
2. Display Flights
3. Search Flights
4. Sort Flights by Price
5. Book Flight
6. Cancel Booking
7. Show All Bookings
8. Exit
Enter your choice: 3
1. Search by Destination
2. Search by Flight ID
1
Enter Destination: Dubai
ID      Source        Destination  Time       Price      Seats
103     Pune          Dubai        12         5000.00    34
```

Figure 6.4: Demonstration of Sort Flights by Price (Bubble Sort)

```
========== FLIGHT RESERVATION SYSTEM ==========
1. Add Flight
2. Display Flights
3. Search Flights
4. Sort Flights by Price
5. Book Flight
6. Cancel Booking
7. Show All Bookings
8. Exit
Enter your choice: 4
? Flights sorted by price (Bubble Sort applied)
```

```
ID      Source          Destination     Time        Price       Seats
102     Pune            Mumbai          11          2000.00     23
101     Pune            Delhi           10          4000.00     12
103     Pune            Dubai           12          5000.00     34
```

Figure 6.5: Demonstration of Booking and Showing Passenger Details

```
========== FLIGHT RESERVATION SYSTEM ==========
1. Add Flight
2. Display Flights
3. Search Flights
4. Sort Flights by Price
5. Book Flight
6. Cancel Booking
7. Show All Bookings
8. Exit
Enter your choice: 5
ID      Source          Destination     Time        Price       Seats
102     Pune            Mumbai          11          2000.00     23
101     Pune            Delhi           10          4000.00     12
103     Pune            Dubai           12          5000.00     34

Enter Flight ID to book: 102
Enter Passenger Name: Swapnil Take
? Booking Confirmed for Swapnil Take on Flight 102
```

```
=========== FLIGHT RESERVATION SYSTEM ===========
1. Add Flight
2. Display Flights
3. Search Flights
4. Sort Flights by Price
5. Book Flight
6. Cancel Booking
7. Show All Bookings
8. Exit
Enter your choice: 6
Enter Passenger Name to cancel booking: Swapnil Take
Booking Cancelled for Swapnil Take
```

7.

Conclusion [42]

The Flight Reservation System successfully demonstrates the core principles of **Data Structures and Algorithms**[43]. It applies fundamental searching (**Linear Search**) and sorting (**Bubble Sort**) algorithms effectively to a real-world data management scenario, reinforcing a practical understanding of computational logic and data manipulation[44]. This project fulfills all defined objectives, providing a functional, console-based application implemented in Java.

8.

Future Scope [45]

The current system serves as a strong foundation, and the following enhancements can be integrated in future versions[46]:

- **GUI Integration:** Develop a user-friendly Graphical User Interface (GUI) using technologies like **JavaFX or Swing**[47].
- **Persistent Storage:** Implement **database connectivity using MySQL** to store flight and passenger data persistently, moving beyond simple in-memory arrays[48].
- **Security:** Addition of **authentication** for different user roles (Admin/User login)[49].
- **Advanced Filtering:** Integration with **date and time filters** for more precise flight searching[50].
- **Efficiency:** Enhancement of search and sort performance by replacing **Linear Search** and **Bubble Sort** with more efficient algorithms like **Binary Search** and **Merge/Quick Sort**[51].
- **Real-Time Updates:** Adding functionality for **real-time flight schedule updates**[52].

9.

References [53]

- Data Structures and Algorithms in Java – Robert Lafore [54]
- Java: The Complete Reference – Herbert Schildt [55]
- SPPU DSA Syllabus (2025 Pattern) [56]
- Oracle Java Documentation: https://docs.oracle.com/en/java/ [57]